Guía de Trabajos Prácticos: Subconsultas en MySQL 8

Práctica intensiva sobre subconsultas escalares, operadores de lista (IN, ANY, ALL), existencia correlacionada (EXISTS, NOT EXISTS) y tablas derivadas en FROM.

Pautas de resolución: Resolver cada consigna en MySQL Workbench respetando estrictamente el tipo de subconsulta solicitado. No utilizar `JOIN` como estrategia principal cuando se pida subconsulta, ni utilizar CTEs (`WITH`) en esta entrega.

01 · ESTRUCTURA DE LA BASE DATOS: ACADEMIA_ONLINE

categorias
`id_categoria (PK), nombre`

instructores
`id_instructor (PK), nombre, apellido, email, ciudad, sueldo_base`

cursos
`id_curso (PK), titulo, id_categoria (FK), id_instructor (FK), precio, horas`

estudiantes
`id_estudiante (PK), nombre, apellido1, apellido2, ciudad, edad`

inscripciones
`id_inscripcion (PK), id_estudiante (FK), id_curso (FK), fecha_inscripcion, monto_abonado`

02 · SCRIPT DE CREACIÓN DDL (MYSQL 8)

```
CREATE DATABASE IF NOT EXISTS academia_online; USE academia_online; CREATE TABLE categorias ( id_categoria INT PRIMARY KEY, nombre VARCHAR(50) NOT NULL ) ENGINE = InnoDB; CREATE TABLE instructores ( id_instructor INT PRIMARY KEY, nombre VARCHAR(80) NOT NULL, apellido VARCHAR(80) NOT NULL, email VARCHAR(100) UNIQUE NOT NULL, ciudad VARCHAR(80) NOT NULL, sueldo_base DECIMAL(10,2) NOT NULL ) ENGINE = InnoDB; CREATE TABLE cursos ( id_curso INT PRIMARY KEY, titulo VARCHAR(120) NOT NULL, id_categoria INT NOT NULL, id_instructor INT, precio DECIMAL(10,2) NOT NULL, horas INT NOT NULL, FOREIGN KEY (id_categoria) REFERENCES categorias(id_categoria), FOREIGN KEY (id_instructor) REFERENCES instructores(id_instructor) ) ENGINE = InnoDB; CREATE TABLE estudiantes ( id_estudiante INT PRIMARY KEY, nombre VARCHAR(80) NOT NULL, apellido1 VARCHAR(80) NOT NULL, apellido2 VARCHAR(80), ciudad VARCHAR(80) NOT NULL, edad INT NOT NULL ) ENGINE = InnoDB; CREATE TABLE inscripciones ( id_inscripcion INT PRIMARY KEY, id_estudiante INT NOT NULL, id_curso INT NOT NULL, fecha_inscripcion DATE NOT NULL, monto_abonado DECIMAL(10,2) NOT NULL, FOREIGN KEY (id_estudiante) REFERENCES estudiantes(id_estudiante), FOREIGN KEY (id_curso) REFERENCES cursos(id_curso) ) ENGINE = InnoDB;
```

03 · SCRIPT DE DATOS INICIALES (DML)

```
INSERT INTO categorias VALUES (1,'Programacion'),(2,'Diseno'),(3,'Negocios'),(4,'Ciberseguridad'),
(5,'Marketing'); INSERT INTO instructores VALUES (1, 'Martin', 'Perez', 'martin.perez@academia.com', 'Mar
del Plata', 180000.00), (2, 'Valeria', 'Gomez', 'valeria.gomez@academia.com', 'Balcarce', 240000.00), (3,
'Lucas', 'Diaz', 'lucas.diaz@academia.com', 'Tandil', 195000.00), (4, 'Sofia', 'Romero',
'sofia.romero@academia.com', 'Mar del Plata', 310000.00), (5, 'Ignacio', 'Torres',
'ignacio.torres@academia.com', 'Miramar', 150000.00); INSERT INTO cursos VALUES (101, 'Java Backend &
Spring Boot', 1, 1, 45000.00, 60), (102, 'Bases de Datos Relacionales y SQL', 1, 1, 38000.00, 40), (103,
'Figma y UI/UX Design', 2, 2, 28000.00, 30), (104, 'Finanzas Personales e Inversiones', 3, 3, 22000.00,
20), (105, 'Ethical Hacking Inicial', 4, 4, 52000.00, 50), (106, 'Python para Ciencia de Datos', 1, 1,
49000.00, 55), (107, 'Gestion Agil de Proyectos (Scrum)', 3, NULL, 19000.00, 15), (108, 'SEO y Estrategia
Digital', 5, NULL, 21000.00, 25); INSERT INTO estudiantes VALUES (1, 'Agustin', 'Suarez', 'Lopez', 'Mar del
Plata', 21), (2, 'Camila', 'Vazquez', NULL, 'Balcarce', 27), (3, 'Federico', 'Molina', 'Rios', 'Mar del
Plata', 34), (4, 'Julieta', 'Castro', NULL, 'Tandil', 22), (5, 'Gonzalo', 'Herrera', 'Paz', 'Necochea',
41), (6, 'Belen', 'Benitez', 'Gimenez', 'Mar del Plata', 29), (7, 'Marcos', 'Alvarez', NULL, 'Mar del
Plata', 19); INSERT INTO inscripciones VALUES (1, 1, 101, '2026-03-01', 45000.00), (2, 1, 102, '2026-03-
05', 38000.00), (3, 2, 103, '2026-03-10', 28000.00), (4, 3, 101, '2026-03-12', 45000.00), (5, 3, 105,
'2026-03-15', 52000.00), (6, 4, 104, '2026-03-18', 22000.00), (7, 6, 102, '2026-03-20', 38000.00), (8, 6,
106, '2026-03-22', 49000.00);
```

04 · EJERCITADOR PRÁCTICO — PARTE 1

Ejercicio 01 · Cursos sobre el precio promedio

ESCALAR EN WHERE

Mostrar `id_curso`, `titulo`, `precio` y `horas` de los cursos cuyo precio sea estrictamente mayor que el precio promedio de todos los cursos de la academia.

Condición: Calcular el promedio con una subconsulta escalar en WHERE.

Ejercicio 02 · Instructor con mayor sueldo base

ESCALAR MÁXIMO

Mostrar `id_instructor`, `nombre`, `apellido` y `sueldo_base` del docente que percibe la mayor remuneración base.

Condición: Emplear `= (SELECT MAX(...) ...)` en la cláusula WHERE.

Ejercicio 03 · Cursos de categorías específicas

SUBCONSULTA IN

Listar `id_curso`, `titulo` y `precio` de aquellos cursos que pertenezcan a las áreas de 'Programacion' o 'Ciberseguridad'.

Condición: Resolver sin JOIN, obteniendo los códigos de categoría mediante IN.

Ejercicio 04 · Cursos sin ninguna inscripción

SUBCONSULTA NOT IN

Mostrar el identificador y título de todos los cursos que hasta el momento no registren ninguna venta/inscripción.

Condición: Resolver utilizando el operador NOT IN sobre la tabla inscripciones.

Ejercicio 05 · Comparativa con ANY y ALL

COMPARACIÓN ANY / ALL

Tomando como conjunto de referencia los precios de los cursos de la categoría 'Negocios': a) Mostrar los cursos cuyo precio sea mayor que ANY de dichos precios; b) Mostrar los cursos cuyo precio sea mayor que ALL de dichos precios.

Condición: Escribir ambas consultas y verificar conceptualmente el porqué de la diferencia en las filas devueltas.

Ejercicio 06 · Tabla derivada con facturación total

FROM (TABLA DERIVADA)

Construir una subconsulta en FROM que agrupe por `id_curso` y calcule el total recaudado (`SUM(monto_abonado)`) y cantidad de alumnos. Desde la consulta principal, filtrar solo aquellos cursos cuya recaudación total supere $60.000.

Condición: La tabla derivada debe poseer alias identificatorio obligatorio.

05 · EJERCITADOR PRÁCTICO — PARTE 2 (CORRELACIONADAS E INTEGRADORAS)

Ejercicio 07 · Estudiantes con actividad confirmada — EXISTS CORRELACIONADO

Mostrar `id_estudiante`, `nombre` y `apellido1` de los alumnos que posean al menos una inscripción en el sistema.

Condición: Emplear `WHERE EXISTS (SELECT 1 FROM inscripciones i WHERE ...)`.

Ejercicio 08 · Instructores sin cursos asignados — NOT EXISTS

Mostrar código, nombre y apellido de aquellos docentes que actualmente no tengan asignado ningún curso a su cargo.

Condición: Resolver obligatoriamente mediante `NOT EXISTS`.

Ejercicio 09 · Cursos superiores al promedio de su categoría — CORRELACIONADA EN WHERE

Listar los cursos cuyo valor de venta sea superior al precio promedio de los cursos pertenecientes a **su misma categoría**.

Condición: Subconsulta correlacionada que vincule el `id_categoria` **de la consulta externa con la interna.**

Ejercicio 10 · Cantidad de cursos por instructor en SELECT — CORRELACIONADA EN SELECT

Mostrar `id_instructor`, `nombre`, `apellido` y una columna calculada denominada `total_cursos`. Aquellos instructores sin cursos asignados deben reflejar el valor 0.

Condición: La columna calculada debe obtenerse mediante una subconsulta correlacionada en la lista del SELECT.

Ejercicio 11 · Filtrado en dos etapas — SUBCONSULTA EN FROM

Generar una tabla derivada en el FROM que seleccione todos los docentes con sueldo base mayor o igual a $180.000. Desde la consulta exterior, filtrar para mostrar exclusivamente a aquellos cuya ciudad sea 'Mar del Plata'.

Ejercicio 12 · Problema integrador multidimensional — DOBLE SUBCONSULTA (SELECT + WHERE)

Mostrar `id_estudiante`, `nombre` y `apellido1` de los alumnos que se hayan inscripto en al menos un curso con precio superior a $40.000 (resuelto con subconsulta en WHERE). Incluir además una columna calculada `total_inscripciones` mediante subconsulta correlacionada en SELECT.

06 · CRITERIOS DE CORRECCIÓN Y LISTA DE REVISIÓN

- **Escalares:** Verificar que la subconsulta utilizada con operadores de comparación (`=`, `>`, `<`) devuelva siempre una única fila y una única columna.
- **Listas (IN / ANY / ALL):** Asegurar que la subconsulta interna proyecte una única columna coincidente en tipo con la columna evaluada.
- **Tablas derivadas en FROM:** Todas las consultas que actúen como tablas derivadas deben contar obligatoriamente con un alias (ej: `) AS t`).
- **Correlacionadas:** Validar el paso de parámetros entre la consulta exterior y la interior; en `EXISTS` / `NOT EXISTS` es una buena práctica utilizar `SELECT 1`.